

Documentazione Progetto Sistemi Operativi: PandOSsh Fase 3

Luca Argentino, Milo Disalvatore, Matteo Mazzetti, Zeno Maccari

17 maggio 2026

1 Gestione Memoria Virtuale (vmSupport.c)

Il modulo `vmSupport.c` implementa il supporto per la memoria virtuale, la gestione dei TLB-Refill e il paging.

1.1 Strutture Dati Globali

Le variabili globali e le strutture definite nel modulo sono:

- `swapPool`: Array di tipo `swap_t` e di dimensione `POOLSIZE` (pari a `UPROCMAX * 2`, ovvero 16 frame) che rappresenta la tabella della Swap Pool per allocare le pagine in memoria fisica.
- `swapSemaphore`: Semaforo utilizzato per garantire la mutua esclusione della Swap Pool tra i vari processi.
- `holdingSwapMutex`: Cella di un'array che tiene traccia, tramite indice basato sull'ASID, di quale processo possiede attualmente il mutex per lo swap.

1.2 Descrizione delle Funzioni

1.2.1 `void initPagetable(pteEntry_t *pageTable, int asid)`

La funzione inizializza la Page Table a 32 voci di un U-proc. Tutte le voci vengono marcate come non valide ($V=0$), abilitando la scrittura ($D=1$) e private rispetto all'ASID ($G=0$); nessuna pagina è presente in RAM finché un page fault non la carica.

Il comportamento è il seguente:

1. **Voci 0–30** (testo e dati): `EntryHi` viene impostato con il VPN ($KUSEG + i$) e l'ASID; `EntryLo` con `DIRTYON` ($D=1, V=0, G=0$).
2. **Voce 31** (stack): `EntryHi` viene impostato con il VPN `0xBFFFF`; il top della pagina di stack è `0xC0000000`, punto di partenza dello SP. `EntryLo` viene impostato con `DIRTYON`.

1.2.2 void uTLB_RefillHandler(void)

Questa funzione è il gestore degli eventi di TLB-Refill: viene invocata quando la traduzione di un indirizzo logico non trova una voce corrispondente nel TLB. Esegue in kernel-mode con gli interrupt disabilitati, usando lo stack del Nucleus (come gli altri handler del Livello 3/Fase 2).

Il comportamento è il seguente:

1. Legge lo stato del processore salvato dal BIOS in BIOSDATAPAGE. Il campo EntryHi di questo stato contiene il VPN dell'indirizzo che ha causato il miss.
2. Ricava l'indice nella Page Table tramite `missingVPN % MAXPAGES`: i VPN del testo/dati (0x80000–0x8001E) producono 0–30, mentre il VPN dello stack (0xBFFFF) produce esattamente 31 per le proprietà dell'aritmetica modulo 32.
3. Carica la voce della Page Table nel TLB tramite le istruzioni `setENTRYHI`, `setENTRYLO` e `TLBWR`.
4. Restituisce il controllo al processo corrente tramite `LDST` sullo stato salvato, in modo che l'istruzione che ha causato il miss venga rieseguita.

1.2.3 void initSwapPool()

La funzione inizializza la tabella della Swap Pool e porta il semaforo associato al valore 1, come richiesto dalla specifica per garantire la mutua esclusione.

Il comportamento è il seguente:

1. Per ciascuno dei POOLSIZE frame, inizializza la voce con:
 - `sw_asid = -1`: segnala che il frame è libero (tutti gli ASID validi sono positivi).
 - `sw_pageNo = -1` e `sw_pte = NULL`: campi non significativi finché il frame è libero.
2. Esegue una `VERHOGEN` su `swapSemaphore`, portandolo da 0 (zero-initialization del C) a 1, inizializzandolo così come semaforo di mutua esclusione.

1.2.4 int getFlashBlock(int vpn) e devreg_t *getFlashRegister(int asid)

1. `getFlashBlock`: Funzione ausiliaria che calcola il blocco corrispondente a un dato VPN. Restituisce 31 per l'indirizzo di stack (0xBFFFF), mentre per gli altri VPN sottrae l'offset base 0x80000.
2. `getFlashRegister`: Calcola l'indirizzo fisico dei registri del Flash Device di un processo in base al suo ASID. Se il device non è valido ritorna NULL, altrimenti restituisce il puntatore alla struttura dei registri calcolata tramite l'offset della linea degli interrupt.

1.2.5 `int readWriteFlashdrive(int asid, int vpn, int phisicalFrame, int op)`

1. Ricava il numero di blocco logico dal VPN e il registro corretto del Flash Device associato al processo tramite il suo ASID.
2. Scrive l'indirizzo del frame fisico iniziale all'interno del campo DATA0 del device.
3. Costruisce il comando unendo il numero del blocco (shiftato di 8 bit) con il codice operativo (`op`), per poi inviare una richiesta sincrona sul campo `COMMAND` tramite una `SYSCALL(DOIO)`.

1.2.6 `void pager()`

1. Recupera il puntatore alla struttura di supporto del processo tramite `SYSCALL(GETSUPPORTPTR)` e verifica la causa del TLB fault dall'eccezione salvata.
2. Se l'eccezione è di tipo TLB-Mod (modifica), il comportamento viene dirottato al gestore delle Program Trap.
3. Acquisisce il mutex sulla Swap Pool tramite `PASSEREN` su `swapSemaphore` e aggiorna il tracker locale `holdingSwapMutex`.
4. Sceglie un frame dalla Swap Pool utilizzando un algoritmo FIFO di tipo round-robin per rimpiazzare una pagina. Prima però controlla che ciò sia necessario: se è presente un frame non occupato utilizziamo quello.
5. Se il frame è occupato da una pagina valida, la invalida rimuovendo il flag `VALIDON` dalla sua Page Table, svuota il TLB e salva il vecchio contenuto nel rispettivo Flash Device tramite `FLASHWRITE`.
6. Legge la nuova pagina dal Flash Device del processo corrente e la carica nel frame fisico scelto tramite `FLASHREAD`. In caso di errore durante l'I/O, rilascia il semaforo e genera una Program Trap.
7. Aggiorna l'entry della Swap Pool salvando il nuovo ASID, il numero di pagina (`missingPageNo`) e ricavando l'indirizzo della rispettiva entry nella Page Table.
8. Modifica l'entry della Page Table appena trovata unendo l'indirizzo fisico del frame e impostando i flag `DIRTYON` e `VALIDON`.
9. Rilascia l'uso del semaforo su `swapSemaphore` (resettando anche `holdingSwapMutex`) e restituisce il controllo ricaricando lo stato per tentare nuovamente l'istruzione `LDST`.

2 Gestione delle Syscall Positive

Questa parte del progetto implementa la gestione delle eccezioni generali del Support Level, alcune syscall positive richieste dagli U-proc, l'inizializzazione

dei processi utente e due programmi utente: la shell e `calc`. I file principali coinvolti sono:

- `sysSupport.c`: gestisce le eccezioni generali del Support Level, distinguendo tra syscall positive e Program Trap.
- `syscalls.c`: implementa le syscall positive `SYS2`, `SYS4`, `SYS5` e `SYS6`, più le funzioni ausiliarie di I/O sul terminale.

2.1 `sysSupport.c`: Gestione delle Eccezioni Generali

Il modulo `sysSupport.c` contiene il gestore generale delle eccezioni del Support Level. Questo gestore riceve il controllo quando il Nucleus effettua il meccanismo di *Pass Up* verso il Support Level per eccezioni non legate direttamente al TLB.

2.1.1 `void generalExceptionHandler()`

La funzione `generalExceptionHandler()` è il punto di ingresso per le eccezioni generali del Support Level. Il suo funzionamento può essere riassunto nei seguenti passi:

- Recupera la `Support Structure` del processo corrente tramite la syscall negativa `GETSUPPORTPTR`.
- Legge il registro `cause` salvato in `sup_exceptState[GENERALEXCEPT]`.
- Estrae il codice dell'eccezione tramite la maschera `CAUSE_EXCCODE_MASK`.
- Se l'eccezione è una syscall, cioè `SYSEXCEPTION` oppure codice 11, passa il controllo a `pos_syscallHandler()`.
- In tutti gli altri casi considera l'eccezione come una Program Trap e invoca `programTrapHandler()`.

Questa funzione quindi separa le syscall positive dalle eccezioni dovute a comportamenti non validi del processo utente.

2.1.2 `void programTrapHandler(support_t *sup)`

La funzione `programTrapHandler()` gestisce le Program Trap. Nel nostro progetto una Program Trap viene trattata come una terminazione ordinata del processo che l'ha causata.

Per questo motivo la funzione richiama direttamente: `sys2(sup)`

In questo modo il processo viene terminato tramite la stessa logica usata dalla syscall positiva `SYS2`.

2.1.3 void pos_syscallHandler(support_t *sup)

La funzione `pos_syscallHandler()` implementa il gestore delle syscall positive, cioè le syscall disponibili agli U-proc.

Il comportamento della funzione è il seguente:

- (a) Recupera lo stato salvato dell'eccezione generale:
`sup->sup_exceptState[GENERALEXCEPT]`
- (b) Legge da `reg_a0` il numero della syscall richiesta.
- (c) Incrementa il program counter di `WORDLEN`, in modo che, al ritorno dalla syscall, il processo riprenda dall'istruzione successiva alla `SYSCALL`. Senza questo incremento il processo rieseguirebbe la stessa syscall in loop.
- (d) Usa uno `switch` sul valore di `a0` per invocare il servizio richiesto:
 - `TERMINATE`: invoca `sys2(sup)`.
 - `WRITETERMINAL`: invoca `sys4(sup)`.
 - `READTERMINAL`: invoca `sys5(sup)`.
 - `EXECUTE`: invoca `sys6(sup)`.
- (e) Se il numero della syscall non è riconosciuto, il processo viene terminato tramite `sys2(sup)`.

2.2 syscalls.c: Syscall Positive

Il modulo `syscalls.c` contiene l'implementazione effettiva di tutte le syscall positive accessibili agli U-proc (`SYS2`, `SYS4`, `SYS5`, `SYS6`) e delle funzioni ausiliarie di I/O sul terminale.

2.2.1 int writeTerminal(char *str, int len)

Funzione che trasmette `len` caratteri della stringa `str` sul terminale 0, carattere per carattere tramite la syscall negativa `D0I0` (`NSYS5`).

Scelta progettuale sul semaforo: il semaforo `termWriteSemaphore` viene acquisito *una sola volta* prima del ciclo di trasmissione e rilasciato al termine. Acquisire e rilasciare il semaforo ad ogni singolo carattere permetterebbe ad un altro U-proc di interporsi tra due caratteri consecutivi, mescolando l'output di stringhe diverse sullo stesso terminale.

Il comportamento è il seguente:

- (a) Acquisisce `termWriteSemaphore` con una P (`PASSEREN`).
- (b) Per ogni carattere da trasmettere:

- Costruisce il comando: `(char < 8) | TRANSMITCHAR`, dove `TRANSMITCHAR = 2`.
 - Invia il comando al registro `transm_command` del terminale tramite `DOIO`; il processo rimane sospeso fino all'interrupt di completamento gestito dal Nucleus.
 - Controlla il byte di stato restituito (bit 7:0). Se diverso da `OKCHARTRANS (5)`, rilascia il semaforo e restituisce il negativo del byte di stato.
- (c) Rilascia `termWriteSemaphore` con una V (VERHOGEN).
- (d) Restituisce il numero di caratteri trasmessi con successo.

2.2.2 static int readTerminal(char *buf)

Funzione che legge una riga dal terminale 0 nel buffer `buf`, carattere per carattere tramite `DOIO (NSYS5)`. La lettura termina al carattere newline (`\n`) o su errore del device.

Per la stessa motivazione di `writeTerminal`, il semaforo `termReadSemaphore` viene acquisito una sola volta prima del ciclo di ricezione: ciò garantisce che l'intera riga venga letta da un solo U-proc senza interleaving.

Il comportamento è il seguente:

- (a) Acquisisce `termReadSemaphore` con una P.
- (b) In un ciclo while:
- Invia il comando `RECEIVECHAR (2)` al registro `recv_command` del terminale tramite `DOIO`.
 - Controlla il byte di stato (bit 7:0). Se diverso da `CHARRECV (5)`, rilascia il semaforo e restituisce il negativo del byte di stato.
 - Estrae il carattere ricevuto dai bit 15:8 del registro di stato.
 - Aggiunge il carattere al buffer e incrementa il contatore.
 - Se il carattere è `\n`, termina il ciclo.
- (c) Rilascia `termReadSemaphore` con una V.
- (d) Restituisce il numero di caratteri ricevuti.

2.2.3 void sys2(support_t *sup)

La funzione `sys2()` implementa la syscall positiva `TERMINATE`, cioè `SYS2`. Questa syscall permette a un U-proc di terminare la propria esecuzione in modo ordinato.

Il comportamento della funzione è il seguente:

- (a) Recupera l'ASID del processo corrente tramite: `sup->sup_asid`

- (b) Controlla se il processo sta mantenendo il mutex sulla Swap Pool tramite l'array `holdingSwapMutex`. Se il processo possiede ancora il mutex:
 - resetta il flag associato all'ASID;
 - rilascia il semaforo `swapSemaphore` tramite `VERHOGEN`.
- (c) Se il processo da terminare è la shell, cioè ha ASID uguale a `SHELL_ASID`, viene eseguita una V su `masterSemaphore`. Questo sblocca l'*Instantiator Process*, che stava aspettando la terminazione della shell.
- (d) Se invece il processo non è la shell, viene eseguita una V su `shellSemaphore`. Questo sblocca la shell, che era rimasta in attesa della terminazione del programma lanciato tramite `SYS6`.
- (e) Come ottimizzazione, la funzione acquisisce `swapSemaphore` e scorre tutta la `swapPool`. Per ogni frame appartenente al processo terminato, cioè con `sw_asid == asid`, imposta `sw_asid = -1`. In questo modo i frame occupati dal processo terminato vengono marcati come liberi.
- (f) Rilascia nuovamente `swapSemaphore`.
- (g) Infine invoca la syscall negativa del Nucleus: `SYSCALL(TERMPROCESS, 0, 0, 0)` che termina effettivamente il processo corrente.

Questa implementazione non si limita quindi a terminare il processo, ma si occupa anche di mantenere coerenti i semafori usati per la sincronizzazione tra shell, processi figli e Instantiator Process.

2.2.4 void sys4(support_t *sup)

Implementa la syscall positiva `WRITETERMINAL` (`SYS4`). Sospende il processo richiedente fino alla trasmissione di una stringa sul terminale associato all'U-proc.

Parametri letti dallo stato salvato (`sup_exceptState[GENERALEXCEPT]`):

- `reg_a1`: indirizzo virtuale della stringa da trasmettere.
- `reg_a2`: lunghezza della stringa.

Il comportamento è il seguente:

- (a) **Validazione indirizzo:** l'intera stringa deve essere contenuta nel segmento utente (`kuseg`), ovvero:

$$\text{str} \geq \text{KUSEG} \text{ e } \text{str} + \text{len} \leq \text{USERSTACKTOP}$$

In caso contrario il processo viene terminato tramite `sys2()`.

- (b) **Validazione lunghezza:** deve essere compresa in `[0, MAXSTRENG]` (128 caratteri). Altrimenti il processo viene terminato.
- (c) Se le validazioni hanno successo, chiama `writeTerminal(str, len)` e salva il risultato in `reg_a0` dello stato salvato.

- (d) Ripristina il processo con `LDST(excState)`: il PC è già stato avanzato di `WORDLEN` da `pos_syscallHandler()`, quindi il processo riprende all'istruzione successiva alla `SYSCALL`.

2.2.5 void sys5(support_t *sup)

Implementa la syscall positiva `READTERMINAL` (`SYS5`). Sospende il processo richiedente fino alla ricezione di una riga dal terminale.

Parametri letti dallo stato salvato:

- `reg_a1`: indirizzo virtuale del buffer di destinazione.

Il comportamento è il seguente:

- (a) **Validazione indirizzo:** il buffer deve trovarsi in `kuseg` (`buf >= KUSEG` || `buf < USERSTACKTOP`). Altrimenti il processo viene terminato.
- (b) Chiama `readTerminal(buf)` e salva il risultato in `reg_a0`.
- (c) Ripristina il processo con `LDST(excState)`.

2.2.6 void sys6(support_t *sup)

La funzione `sys6()` implementa la syscall positiva `EXECUTE`, cioè `SYS6`. Questa syscall consente alla shell di creare un nuovo U-proc dato il suo ASID.

Il comportamento della funzione è il seguente:

- (a) Recupera lo stato salvato dell'eccezione generale:
`sup->sup_exceptState[GENERALEXCEPT]`
- (b) Legge da `reg_a1` l'ASID del processo da creare.
- (c) Verifica che il chiamante sia effettivamente la shell. Se `sup->sup_asid` è diverso da `SHELL_ASID`, la richiesta non è valida e il processo viene terminato tramite `sys2()`.
- (d) Verifica che l'ASID richiesto sia valido:
 - deve essere maggiore di 0;
 - deve essere minore o uguale a `UPROCMAX`;
 - non deve essere uguale a `SHELL_ASID`.
- (e) Se i controlli sono superati, crea il nuovo U-proc tramite: `createUProc(asid)`
- (f) Se la creazione fallisce e viene restituito `NOPROC`, il processo chiamante viene terminato.
- (g) Dopo aver creato il nuovo processo, la shell viene bloccata con una P su `shellSemaphore`. In questo modo la shell rimane sospesa fino alla terminazione del processo figlio.

- (h) Quando il processo figlio termina, la `sys2()` del figlio esegue una `V` su `shellSemaphore`, sbloccando la shell.
- (i) Infine viene ricaricato lo stato salvato tramite `LDST(excState)`, permettendo alla shell di riprendere l'esecuzione.

Questa syscall realizza quindi un modello di esecuzione sincrona: la shell lancia un programma e resta bloccata fino alla sua terminazione.

3 Inizializzazione degli U-proc

3.1 `initProc.c`: Inizializzazione del Support Level

Il modulo `initProc.c` contiene l'*Instantiator Process*, cioè la funzione `test()`, e le strutture globali necessarie alla fase 3.

3.1.1 Variabili globali

Le principali variabili globali definite nel modulo sono:

- `masterSemaphore`: semaforo inizializzato a 0. Viene usato per bloccare l'Instantiator Process fino alla terminazione della shell.
- `shellSemaphore`: semaforo inizializzato a 0. Viene usato per bloccare la shell mentre un programma lanciato tramite `SYS6` è in esecuzione.
- `termWriteSemaphore`: semaforo di mutua esclusione per la scrittura sul terminale.
- `termReadSemaphore`: semaforo di mutua esclusione per la lettura dal terminale.
- `supportPool[UPROCMAX]`: array statico di `support_t`, usato per assegnare una Support Structure a ogni U-proc.

3.1.2 `int createUProc(int asid)`

La funzione `createUProc()` crea un nuovo U-proc a partire dal suo ASID.

Il suo funzionamento è il seguente:

- (a) Controlla che l'ASID sia valido. Se l'ASID è minore o uguale a 0, oppure maggiore di `UPROCMAX`, viene invocata `PANIC()`.
- (b) Dichiarare uno stato iniziale del processore: `state_t procState`
- (c) Ricava la Support Structure associata all'ASID tramite: `&supportPool[asid-1]`

- (d) Inizializza lo stato del processo e la sua Support Structure tramite:
`initUProc(asid, &procState, support)`
- (e) Crea il processo tramite la syscall negativa `CREATEPROCESS`, passando:
 - lo stato iniziale del processore;
 - la priorità `UPROC_PRIORITY`;
 - il puntatore alla Support Structure.
- (f) Restituisce il PID del processo creato, oppure `NOPROC` in caso di errore.

3.1.3 void test()

La funzione `test()` rappresenta l'Instantiator Process della fase 3. Il suo compito è inizializzare il Support Level, avviare la shell e poi attendere la sua terminazione.

Il comportamento della funzione è il seguente:

- (a) Inizializza `masterSemaphore` e `shellSemaphore` a 0.
- (b) Inizializza i semafori del terminale:
 - `termWriteSemaphore = 1;`
 - `termReadSemaphore = 1.`
- (c) Invoca `initSwapPool()`, che inizializza le strutture dati per la memoria virtuale e la Swap Pool.
- (d) Crea la shell tramite: `createUProc(SHELL_ASID)`
- (e) Se la creazione della shell fallisce, invoca `PANIC()`.
- (f) Esegue una P su `masterSemaphore`. In questo modo l'Instantiator Process resta bloccato finché la shell non termina.
- (g) Quando la shell termina, la sua `sys2()` esegue una V su `masterSemaphore`; a quel punto `test()` riprende l'esecuzione.
- (h) Infine termina se stesso tramite: `SYSCALL(TERMPROCESS, 0, 0, 0)`

La terminazione di `test()` porta il sistema alla conclusione ordinata, lasciando al Nucleus il compito di effettuare l'`HALT` quando non restano più processi attivi.

4 Testers

4.1 shell.c: Shell Utente

Il file `shell.c` implementa un semplice interprete di comandi testuale. La shell legge un comando da terminale, lo confronta con una tabella statica di programmi disponibili e, se il comando è valido, invoca la syscall `EXECUTE`.

4.1.1 Mappatura tra comandi e ASID

La shell mantiene una tabella statica di associazione tra nome del programma e ASID:

- `fibEight` → ASID 2
- `echo` → ASID 3
- `fibEleven` → ASID 4
- `uname` → ASID 5
- `date` → ASID 6
- `sl` → ASID 7
- `calc` → ASID 8

Questa mappatura permette alla shell di convertire il nome digitato dall'utente nell'ASID richiesto dalla syscall `SYS6`.

4.1.2 `void trimSpaces(char *s)`

La funzione `trimSpaces()` rimuove gli spazi iniziali e finali dal comando inserito dall'utente.

4.1.3 `void chkExitAndTerminate(char *buff)`

La funzione `chkExitAndTerminate()` controlla se il comando inserito è: `exit`
In caso positivo invoca: `SYSCALL(TERMINATE, 0, 0, 0)`
terminando la shell. La terminazione della shell sblocca poi l'Instantiator Process tramite il meccanismo implementato in `sys2()`.

4.1.4 `void main()`

La funzione `main()` della shell contiene il ciclo principale dell'interprete.
Il funzionamento è il seguente:

- (a) Stampa il prompt: `>`
- (b) Legge una riga da terminale tramite `READTERMINAL`.
- (c) Sostituisce il carattere newline finale con il terminatore di stringa `\0`.
- (d) Rimuove gli spazi iniziali e finali tramite `trimSpaces()`.
- (e) Controlla se il comando è `exit`. In tal caso termina la shell.
- (f) Se il comando è vuoto, ricomincia il ciclo.

- (g) Cerca il comando nella tabella statica dei programmi.
- (h) Se il comando viene trovato, invoca: `SYSCALL(EXECUTE, excAsid, 0, 0)`
dove `excAsid` è l'ASID associato al programma.
- (i) Se il comando non viene trovato, stampa: `command not found :<`

La shell quindi fornisce un'interfaccia minimale per testare gli U-proc e le syscall positive del Support Level.

4.2 `calc.c`: Programma Calcolatrice

Il file `calc.c` implementa un programma utente che esegue operazioni aritmetiche semplici tra due numeri a una cifra. Il programma viene avviato dalla shell tramite il comando: `calc`

4.2.1 `void deleteSpaces(char *expr, int len, char *newExpr)`

La funzione `deleteSpaces()` copia l'espressione di input in una nuova stringa rimuovendo tutti gli spazi

4.2.2 `int asciiToInt(char *expr, int *first, char *opr, int *second, int len)`

La funzione `asciiToInt()` effettua il parsing dell'espressione inserita dall'utente.

Il comportamento è il seguente:

- (a) Rimuove gli spazi dall'espressione tramite `deleteSpaces()`.
- (b) Converte il primo carattere in intero sottraendo il valore ASCII di `'0'`.
- (c) Verifica che il primo valore sia compreso tra 0 e 9. In caso contrario stampa un messaggio di errore e restituisce -1.
- (d) Controlla che l'operatore sia uno tra:

`+, -, *, /`
- (e) Converte il secondo carattere in intero.
- (f) Verifica che anche il secondo valore sia compreso tra 0 e 9.
- (g) Se tutti i controlli hanno successo, restituisce 1.

4.2.3 `int calculate(int num1, char opr, int num2)`

La funzione `calculate()` esegue l'operazione aritmetica richiesta.

Nel caso di divisione per zero, il programma stampa un messaggio di errore e termina tramite: `SYSCALL(TERMINATE, 0, 0, 0)`

4.2.4 `void main()`

La funzione `main()` di `calc` rappresenta il flusso principale del programma.

Il comportamento è il seguente:

- (a) Stampa il messaggio: `insert expression`
- (b) Legge l'espressione da terminale tramite `READTERMINAL`.
- (c) Invoca `asciiToInt()` per convertire l'input in:
 - primo operando;
 - operatore;
 - secondo operando.
- (d) Se il parsing fallisce, termina il processo tramite `TERMINATE`.
- (e) Calcola il risultato tramite `calculate()`.
- (f) Converte il risultato in caratteri ASCII e lo inserisce in `res_buffer`.
- (g) Stampa il risultato tramite `WRITETERMINAL`.
- (h) Termina il processo tramite: `SYSCALL(TERMINATE, 0, 0, 0)`